Name: Prathamesh Balaji Patil

Roll: 11

Div: TY A(AIML)

Assignment Non Linear

## Experiment: Banknote Authentication using Non-Linear SVM (RBF Kernel)

**Objective:** Classify banknote samples as genuine (`class = 0`) or counterfeit (`class = 1`) using a Non-Linear Support Vector Machine with an RBF kernel.

This notebook uses the same 1,372-row Banknote Authentication dataset as the Linear SVM practical, but trains a different model. It also creates a **separate Flask application and a distinct HTML interface** for the Non-Linear SVM practical.

**Input features:** `variance`, `skewness`, `curtosis`, `entropy`
**Target:** `class`

## 1. Install / import libraries

```
!pip -q install flask joblib scikit-learn pandas
```

```
import os
import shutil
import joblib
import pandas as pd

from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

## 2. Upload the dataset CSV

```
uploaded = files.upload()  # Select banknote_authentication_nonlinear_svm.csv
dataset_path = next(iter(uploaded.keys()))

df = pd.read_csv(dataset_path)
print("Dataset shape:", df.shape)
display(df.head())
print("\\nMissing values per column:")
display(df.isnull().sum())
```

Choose files banknote_a…ear_svm.csv

**banknote_authentication_nonlinear_svm.csv**(text/csv) - 45073 bytes, last modified: 13/10/2026 - 100% done

```
Saving banknote_authentication_nonlinear_svm.csv to banknote_authentication_nonlinear_svm.csv
Dataset shape: (1372, 5)
```

| | variance | skewness | curtosis | entropy | class |
|---|---|---|---|---|---|
| **0** | 3.62160 | 8.6661 | -2.8073 | -0.44699 | 0 |
| **1** | 4.54590 | 8.1674 | -2.4586 | -1.46210 | 0 |
| **2** | 3.86600 | -2.6383 | 1.9242 | 0.10645 | 0 |
| **3** | 3.45660 | 9.5228 | -4.0112 | -3.59440 | 0 |
| **4** | 0.32924 | -4.4552 | 4.5718 | -0.98880 | 0 |

```
\nMissing values per column:
```

| | 0 |
|---|---|
| **variance** | 0 |
| **skewness** | 0 |
| **curtosis** | 0 |
| **entropy** | 0 |
| **class** | 0 |

**dtype:** int64

## 3. Prepare features and target

```
expected_columns = ["variance", "skewness", "curtosis", "entropy", "class"]
missing_columns = [column for column in expected_columns if column not in df.columns]
if missing_columns:
    raise ValueError(f"Dataset is missing columns: {missing_columns}")

df = df[expected_columns].dropna()
X = df[["variance", "skewness", "curtosis", "entropy"]]
y = df["class"].astype(int)

print("Clean dataset shape:", df.shape)
print("Class distribution:")
display(y.value_counts().sort_index().rename(index={0: "Genuine (0)", 1: "Counterfeit (1)"}))
```

```
Clean dataset shape: (1372, 5)
Class distribution:
```

| class | count |
|---|---|
| **Genuine (0)** | 762 |
| **Counterfeit (1)** | 610 |

**dtype:** int64

## 4. Split data into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 1097
Testing samples: 275
```

## 5. Train the Non-Linear SVM model (RBF kernel)

```
nonlinear_model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=1.0, gamma="scale")
)
```

```
nonlinear_model.fit(X_train, y_train)
print("Non-Linear SVM (RBF) training completed.")
```

```
Non-Linear SVM (RBF) training completed.
```

## 6. Evaluate the model

```
y_pred = nonlinear_model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\\nClassification report:")
print(classification_report(y_test, y_pred, target_names=["Genuine (0)", "Counterfeit (1)"]))
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 1.0000
\nClassification report:
                precision    recall  f1-score   support

    Genuine (0)       1.00      1.00      1.00       153
Counterfeit (1)       1.00      1.00      1.00       122

       accuracy                           1.00       275
      macro avg       1.00      1.00      1.00       275
   weighted avg       1.00      1.00      1.00       275

Confusion matrix:
[[153   0]
 [  0 122]]
```

## 7. Test a sample prediction

```
sample = pd.DataFrame(
    [[3.6216, 8.6661, -2.8073, -0.44699]],
    columns=["variance", "skewness", "curtosis", "entropy"]
)
sample_class = int(nonlinear_model.predict(sample)[0])
print("Sample features:")
display(sample)
print("Predicted class:", sample_class)
print("Prediction:", "Genuine banknote" if sample_class == 0 else "Counterfeit banknote")
```

```
Sample features:
```

| | variance | skewness | curtosis | entropy |
|---|---|---|---|---|
| **0** | 3.6216 | 8.6661 | -2.8073 | -0.44699 |

```
Predicted class: 0
Prediction: Genuine banknote
```

## 8. Save the trained Non-Linear SVM model

```
model_filename = "nonlinear_svm_banknote_model.pkl"
joblib.dump(nonlinear_model, model_filename)
print("Saved model:", model_filename)

files.download(model_filename)
```

```
Saved model: nonlinear_svm_banknote_model.pkl
```

## HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>RBF SVM | Banknote Lab</title>
<style>
:root{--ink:#182230;--muted:#687588;--paper:#ffffff;--line:#dce4ec;--accent:#0f766e;--accent2:#115e59}
*{box-sizing:border-box}
body{margin:0;min-height:100vh;background:#edf2f4;color:var(--ink);font-family:Inter,Segoe UI,Arial,sans-serif;padding:28p
.shell{max-width:900px;margin:auto}
```

```
.top{display:flex;align-items:center;justify-content:space-between;gap:18px;margin-bottom:24px}
.brand{font-size:13px;letter-spacing:2px;text-transform:uppercase;font-weight:800;color:var(--accent)}
.kernel{background:#d7f3ee;color:#115e59;border-radius:8px;padding:9px 12px;font-size:12px;font-weight:800}
h1{font-size:clamp(28px,5vw,43px);letter-spacing:-1.4px;line-height:1.08;margin:0 0 12px}
.lead{color:var(--muted);line-height:1.6;margin:0 0 26px;max-width:650px}
.layout{display:grid;grid-template-columns:1.15fr .85fr;gap:20px}
.panel{background:var(--paper);border:1px solid var(--line);border-radius:16px;padding:25px;box-shadow:0 12px 28px #172b3a(
h2{font-size:18px;margin:0 0 7px}
.sub{color:var(--muted);font-size:13px;line-height:1.5;margin:0 0 20px}
.field{margin-bottom:15px}
label{display:block;font-size:13px;font-weight:700;margin-bottom:7px}
input{width:100%;border:1px solid var(--line);border-radius:9px;padding:12px;font-size:15px;background:#fbfdfe}
input:focus{outline:2px solid #99d9d0;border-color:var(--accent)}
.actions{display:flex;gap:10px;margin-top:22px}
button,.reset{border:0;border-radius:9px;padding:12px 15px;font-weight:800;font-size:14px;text-decoration:none;cursor:point
button{background:var(--accent);color:white;flex:1}
button:hover{background:var(--accent2)}
.reset{background:#edf2f4;color:var(--ink)}
.side{background:#172b3a;color:#f7fbfc;border-radius:16px;padding:25px;display:flex;flex-direction:column;justify-content:s
.side h2{font-size:21px;line-height:1.3;margin-bottom:12px}
.side p{color:#c5d2db;font-size:14px;line-height:1.65}
.steps{border-top:1px solid #3d5260;margin-top:22px;padding-top:18px}
.steps div{display:flex;gap:12px;margin:13px 0;color:#e3ebef;font-size:13px;line-height:1.45}
.num{color:#73d4c5;font-weight:900}
.result{grid-column:1/-1;padding:20px;border-radius:13px}
.genuine{background:#e0f7ec;border:1px solid #9bd9b7;color:#14532d}
.counterfeit{background:#fff0e9;border:1px solid #f2bba5;color:#9a3412}
.error{grid-column:1/-1;background:#fff7ed;border:1px solid #fed7aa;border-radius:12px;padding:15px;color:#9a3412}
.result h2{margin-bottom:8px}
.result p{margin:5px 0;line-height:1.5}
footer{text-align:center;color:var(--muted);font-size:12px;margin:22px 0 4px}
@media(max-width:700px){.layout{grid-template-columns:1fr}.side{min-height:280px}.top{align-items:flex-start;flex-directior
</style>
</head>
<body>
<main class="shell">
  <div class="top"><div class="brand">Banknote Lab / ML Experiment</div><div class="kernel">NON-LINEAR SVM · RBF KERNEL</di
  <h1>Banknote<br>Authentication</h1>
  <p class="lead">Enter the four statistical features to classify a sample with the trained Radial Basis Function Support \
  <div class="layout">
    <section class="panel">
      <h2>Sample measurements</h2>
      <p class="sub">Use numerical feature values from the banknote authentication dataset.</p>
      <form method="POST" action="/">
        <div class="field"><label for="variance">Variance</label><input id="variance" name="variance" type="number" step="a
        <div class="field"><label for="skewness">Skewness</label><input id="skewness" name="skewness" type="number" step="a
        <div class="field"><label for="curtosis">Curtosis</label><input id="curtosis" name="curtosis" type="number" step="a
        <div class="field"><label for="entropy">Entropy</label><input id="entropy" name="entropy" type="number" step="any"
        <div class="actions"><button type="submit">Run RBF Prediction</button><a class="reset" href="/">Clear</a></div>
      </form>
    </section>
    <aside class="side">
      <div>
        <h2>A flexible decision boundary for complex patterns.</h2>
        <p>This experiment uses an RBF kernel. Inputs are standardized inside the saved model pipeline before classificatic
      </div>
      <div class="steps">
        <div><span class="num">01</span><span>Enter four features</span></div>
        <div><span class="num">02</span><span>Flask sends the values to the trained model</span></div>
        <div><span class="num">03</span><span>View the predicted class</span></div>
      </div>
    </aside>
    {% if result %}
    <section class="result {{ result.kind }}" role="status">
      <h2>{{ result.label }}</h2>
      <p>{{ result.detail }}</p>
      <p><strong>Predicted class:</strong> {{ result['class'] }} (0 = genuine, 1 = counterfeit)</p>
    </section>
    {% endif %}
    {% if error %}<div class="error" role="alert">{{ error }}</div>{% endif %}
  </div>
  <footer>Academic demonstration only. This prediction is not certified banknote verification.</footer>
</main>
</body>
</html>
```

## Flask Code:

```
from pathlib import Path
```

```
import joblib
import pandas as pd
from flask import Flask, render_template, request

app = Flask(__name__)
BASE_DIR = Path(__file__).resolve().parent
MODEL_PATH = BASE_DIR / "nonlinear_svm_banknote_model.pkl"

FEATURES = ["variance", "skewness", "curtosis", "entropy"]
model = joblib.load(MODEL_PATH)


@app.route("/", methods=["GET", "POST"])
def home():
    result = None
    error = None
    values = {feature: "" for feature in FEATURES}

    if request.method == "POST":
        try:
            data = {}
            for feature in FEATURES:
                raw = request.form.get(feature, "").strip()
                values[feature] = raw
                if not raw:
                    raise ValueError("Please fill in all four feature fields.")
                data[feature] = float(raw)

            sample = pd.DataFrame([data], columns=FEATURES)
            predicted_class = int(model.predict(sample)[0])

            result = {
                "class": predicted_class,
                "label": "GENUINE BANKNOTE" if predicted_class == 0 else "COUNTERFEIT BANKNOTE",
                "detail": (
                    "The RBF SVM classified this sample as genuine."
                    if predicted_class == 0
                    else "The RBF SVM classified this sample as counterfeit."
                ),
                "kind": "genuine" if predicted_class == 0 else "counterfeit",
            }
        except ValueError as exc:
            error = str(exc) if str(exc).startswith("Please fill") else "Enter valid numeric values in all fields."
        except Exception:
            app.logger.exception("Prediction error")
            error = "Prediction failed. Confirm the model file and installed dependencies."

    return render_template("index.html", result=result, error=error, values=values)


if __name__ == "__main__":
    app.run(debug=True)
```